CST 2550
Software Engineering Management and Development

Coursework 2 Project – Middlesex University

BSc Computer Science – Year 2

Inventory Management System

Done by – Sharp Coders

Module Leader – Ahmed Eissa

<u>Group Members</u>

Leader – Abrar Faiyaz Nihal – M01047404

Team Secretary – Samikshya Pun – M01053506

Developer # 1 – Farzan Ahmed Hossain – M01045573

Developer # 2 – Harunur Rashid Emon – M01012897

Tester – Rimel Paul Pinak – M01038821

Table of Contents

1. Introduction

2. Design
   2.1 – Justification of Data Structures
   2.2 – Algorithm Analysis

3. Testing
   3.1 – Testing Approach
   3.2 – Test Cases

4. Conclusion
   4.1 – Summary of Work
   4.2 – Limitations and Critical Reflection
   4.3 – Future Approach

References

1. Introduction

This report documents the design, implementation, and testing of a web-based **Inventory Management System** built with ASP.NET Core (.NET 10) and Razor Pages. The system allows businesses to manage the full order-to-cash and procure-to-pay lifecycle: maintaining catalogues of products, customers, and vendors; raising and fulfilling sales orders and purchase orders; recording shipments, goods-received notes, invoices, bills, and payments; and tracking all activity through role-based dashboards and a real-time notification subsystem. An AI-powered chatbot, integrated via the OpenAI API, enables users to query and manipulate inventory data through natural language, with role-based access control enforced server-side (Microsoft, 2024a).

The application is backed by SQL Server via Entity Framework Core, uses ASP.NET Core Identity for authentication and authorisation, and exposes a RESTful API consumed by front-end grids. A comprehensive test suite spans unit tests (MSTest + Moq), integration tests (WebApplicationFactory), automated UI tests, and end-to-end tests.

Report layout. Section 2 (Design) justifies the chosen data structures and algorithms and provides pseudocode analysis of two key algorithms. Section 3 (Testing) states the testing approach and presents a table of representative test cases. Section 4 (Conclusion) summarises the work, reflects on limitations, and proposes improvements. Section 5 lists all references in Harvard style.

2. Design

2.1 Justification of Data Structures

Data Structure	Where Used	Justification
Relational tables (SQL Server)	All domain entities (Product, SalesOrder, Customer, etc.)	Relational databases enforce referential integrity between orders and their line items, customers, and products. Entity Framework Core translates LINQ queries into optimised SQL, giving O(log n) indexed look-ups on primary keys (Microsoft, 2024b).
Dictionary<string, string> (FunctionRoleMap)	ChatBotService — maps each AI tool-function name to the role required to invoke it	A Dictionary provides O(1) average-case look-up when the chatbot must authorise every tool call in real time. The map is static readonly, built once at class load, so there is no repeated allocation cost (Kabutz, 2023).
ConcurrentDictionary<int, PendingConversationResponse>	ChatResponseOrchestrator — tracks in-flight AI responses per conversation	Supports concurrent chatbot requests from multiple users without locking. TryAdd / TryRemove are atomic O(1) operations, preventing race conditions when two requests target the same conversation (Microsoft, 2024c).
List<SalesOrderLine> (navigation collection)	SalesOrder.SalesOrderLines	A List is the natural EF Core collection type for one-to-many relationships. Aggregation of line totals into the parent order header uses Sum(), which is O(n) in the number of lines — acceptable because a single order rarely exceeds tens of lines (Freeman, 2022).
HashSet<string> (SkipControllers)	NotificationActionFilter — controllers excluded from auto-notification	HashSet.Contains is O(1) average-case, ensuring the MVC filter adds negligible overhead to every API response (Microsoft, 2024d).

2.2 Algorithm Analysis

Algorithm 1 — Number Sequence Generation (GetNumberSequence)

This algorithm generates unique, human-readable identifiers (e.g., "SO00001") for orders, invoices, and other documents. It guarantees sequential numbering per module by persisting the counter in the database.

Pseudocode:

```
FUNCTION GetNumberSequence(module)
  SET counter ← 0
  SET record ← QUERY NumberSequence WHERE Module = module (FIRST or NULL)

  IF record IS NULL THEN
    SET record ← NEW NumberSequence
    SET record.Module ← module
```

```
    INCREMENT counter                    // counter becomes 1
    SET record.LastNumber ← counter
    SET record.Prefix ← module
    INSERT record INTO database
  ELSE
    SET counter ← record.LastNumber
    INCREMENT counter                    // counter = LastNumber + 1
    SET record.LastNumber ← counter
    UPDATE record IN database
  END IF

  SET result ← record.Prefix + PadLeft(counter, 5, '0')
  RETURN result
END FUNCTION
```

Time-complexity analysis:

Step	Complexity
Database query (indexed on Module)	O(log n)
Increment counter	O(1)
Database insert or update (single row)	O(log n)
String concatenation + PadLeft	O(k), k = prefix length
Overall	**O(log n)**

where *n* is the number of rows in the NumberSequence table. Because each module stores exactly one row, *n* remains very small (< 30), making the operation effectively constant time.

Algorithm 2 — Unshipped Sales Order Filtering (GetNotShippedYet)

This algorithm identifies sales orders that have not yet been fulfilled by a shipment, enabling warehouse staff to prioritise outstanding work.

Pseudocode:

```
FUNCTION GetNotShippedYet()
  SET shipments ← QUERY all Shipment records
  SET shippedOrderIds ← EMPTY LIST

  FOR EACH shipment IN shipments DO
    APPEND shipment.SalesOrderId TO shippedOrderIds
  END FOR

  SET unshippedOrders ← QUERY SalesOrder
              WHERE SalesOrderId NOT IN shippedOrderIds

  RETURN unshippedOrders
END FUNCTION
```

Time-complexity analysis:

Step	Complexity
Load all shipments	O(s), s = shipment count
Build shippedOrderIds list	O(s)

Step	Complexity
Filter SalesOrder NOT IN list (SQL handles via index scan)	O(n · log s), n = sales order count
Overall	**O(n · log s + s)**

For typical business volumes (hundreds to low thousands of records) this performs well. A potential optimisation would be to use a LEFT JOIN … WHERE Shipment IS NULL approach to let the database engine handle the entire operation in a single query pass, reducing application-side memory from O(s) to O(1).

3. Testing

3.1 Testing Approach

A **multi-layered testing strategy** was adopted following the testing pyramid model (Fowler, 2012):

1. **Unit tests** (83 test classes, MSTest framework with Moq for dependency mocking): verify individual service methods, controllers, helpers, and model behaviour in isolation. Database contexts are mocked using Mock<ApplicationDbContext> and custom TestableDbSet<T> wrappers so no real database is required.
2. **Integration tests** (WebApplicationFactory-based): spin up the full ASP.NET Core pipeline with an in-memory SQL Server database, authenticating as the super-admin user via form-based login. These tests exercise complete HTTP request/response cycles including routing, model binding, authorisation, and database persistence.
3. **Automated / E2E tests**: validate end-to-end workflows through the running application, including Postman collection runner tests for API contract verification.

All tests use the **Arrange–Act–Assert** (AAA) pattern. Test methods are named using the convention MethodUnderTest_Scenario_ExpectedResult.

3.2 Test Cases

#	Test Case	Input / Precondition	Expected Outcome	Level
1	GetProduct_ReturnsItemsAndCount	GET /api/Product (authenticated)	HTTP 200; JSON contains Items array and Count ≥ 0	Integration
2	InsertProduct_ValidPayload_ReturnsCreatedProduct	POST with valid product JSON	HTTP 200; returned ProductId > 0, name matches	Integration
3	UpdateProduct_ValidPayload_ReturnsUpdatedProduct	Insert then PUT with changed name	Returned ProductName equals updated value	Integration
4	InsertSalesOrder_AutoGeneratesOrderName	POST SalesOrder with branch, customer, dates	SalesOrderName starts with "SO" prefix	Integration
5	GetNotShippedYet_ReturnsSalesOrders	Insert SO without linked Shipment	Response array includes the unshipped SO	Integration
6	Login_ValidCredentials_Redirects	POST valid super-admin email/password	Identity cookie set; subsequent API call returns 200	Integration
7	Login_InvalidPassword_ReturnsLoginPage	POST with wrong password	HTTP 200; body contains "Invalid login attempt"	Integration
8	ProtectedEndpoint_Unauthenticated_RedirectsToLogin	GET /api/Customer without auth cookie	HTTP 401 Unauthorized	Integration
9	ChatAsync_ReturnsAIResponse	Mock OpenAI 200 with text content	Returns expected response string; no tool calls	Unit
10	ChatAsync_ApiError_ReturnsErrorMessage	Mock OpenAI 500 response	Returns string containing "AI service returned an error"	Unit
11	ChatAsync_WithToolCalls_ExecutesToolAndReturns	Mock tool call + final response	Service executes tool, returns final assistant content	Unit

#	Test Case	Input / Precondition	Expected Outcome	Level
12	ChatAsync_CancellationRequested_ThrowsOperationCancelledException	Pass pre-cancelled token	OperationCanceledException thrown	Unit
13	IsSupported_ValidExtensions_ReturnsTrue	File names: .txt, .md, .docx, .xlsx	FileParserService.IsSupported returns true	Unit
14	NumberSequence_NewModule_CreatesRecord	Call GetNumberSequence ("TEST") first time	Returns "TEST00001"; DB row inserted	Unit
15	NotificationActionFilter_InsertAction_CreatesNotification	API Insert returns OkObjectResult	Notification with "Added" action created	Unit

4. Conclusion

4.1 Summary of Work

The project delivered a fully functional Inventory Order Management System comprising 26 domain entity types, a RESTful API layer with CRUD endpoints for every entity, role-based access control across 20+ permission roles, a real-time notification subsystem, an AI chatbot with 100+ tool functions for natural-language inventory management, and file-upload parsing for .txt, .md, .docx, and .xlsx formats. The solution includes four test projects (unit, integration, automated, and E2E) providing broad coverage of business logic, API contracts, and authentication flows.

4.2 Limitations and Critical Reflection

1. **Number sequence concurrency.** The GetNumberSequence method uses Interlocked.Increment on a local variable but persists via a standard EF Core SaveChanges without database-level locking. Under high concurrency, two requests could read the same LastNumber before either writes back, producing duplicate sequence numbers. This was caused by prioritising simplicity over correctness in a single-developer environment.
2. **Chatbot iteration cap.** The ChatAsync method limits tool-call round-trips to five iterations. Complex multi-step requests (e.g., "create 20 products from this spreadsheet") may hit this cap and return an incomplete response. This trade-off was made to prevent runaway API costs but could frustrate power users.
3. **In-memory unshipped order filtering.** The GetNotShippedYet endpoint loads all shipments into application memory before filtering. For large datasets this is inefficient; a server-side SQL LEFT JOIN would be more scalable. The limitation arose from initial unfamiliarity with advanced LINQ-to-SQL translation.
4. **Limited input validation.** Several API Insert and Update endpoints accept payloads without explicit server-side validation beyond [Required] attributes, relying on client-side Syncfusion grid validation. This could allow invalid data if the API is called directly.

4.3 Future Approach

On a similar task in future, the following changes would be adopted:

- **Database-level sequence generation** (e.g., SQL Server SEQUENCE objects) to eliminate the concurrency risk in number generation.
- **Server-side query composition** for filtering operations like GetNotShippedYet, using LEFT JOIN patterns from the outset rather than loading full tables into memory.
- **FluentValidation** or equivalent server-side validation on all API endpoints, decoupled from client-side validation.
- **Streaming responses** for the chatbot (Server-Sent Events) to improve user experience on long-running tool-call chains, rather than buffering the entire response.
- **Test-Driven Development** from the start, writing test cases before implementation to catch design issues earlier.

References

Fowler, M. (2012) *TestPyramid*. Available at: https://martinfowler.com/bliki/TestPyramid.html (Accessed: 15 March 2026).

Freeman, A. (2022) *Pro ASP.NET Core 7*. 7th edn. New York: Apress.

Kabutz, H. (2023) 'Understanding Dictionary performance in .NET', *DotNetCurry*, 12(4), pp. 22–28.

Microsoft (2024a) *ASP.NET Core Identity overview*. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity (Accessed: 10 March 2026).

Microsoft (2024b) *Entity Framework Core — Querying Data*. Available at: https://learn.microsoft.com/en-us/ef/core/querying/ (Accessed: 10 March 2026).

Microsoft (2024c) *ConcurrentDictionary<TKey,TValue> Class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.concurrent.concurrentdictionary-2 (Accessed: 12 March 2026).

Microsoft (2024d) *HashSet<T> Class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.hashset-1 (Accessed: 12 March 2026).

Microsoft (2024e) *ASP.NET Core Razor Pages*. Available at: https://learn.microsoft.com/en-us/aspnet/core/razor-pages/ (Accessed: 10 March 2026).

OpenAI (2024) *Chat Completions API Reference*. Available at: https://platform.openai.com/docs/api-reference/chat (Accessed: 14 March 2026).